

Unit Testing (cont.)

Course Code: CSC4133

Course Title: Software Quality and Testing



Dept. of Computer Science
Faculty of Science and Technology

Lecturer No:	14	Week No:	8	Semester:	
Lecturer:	<i>Prof. Dr. kamruddin Nur, kamruddin@aiub.edu</i>				

Lecture Outline



- Dynamic Unit Testing
- Mutation Testing
- Debugging
- Unit Testing in eXtreme Programming
- Tools For Unit Testing

Objectives and Outcomes



- **Objectives:** To understand the dynamic unit testing, mutation testing, debugging; to understand how to conduct unit testing in eXtreme programming (XP); to know the tools for unit testing.
- **Outcomes:** Students are expected to be able to explain dynamic unit testing, be able to explain mutation testing and debugging; be able to explain how to conduct unit testing in XP; be able to explain different tools for unit testing.

Dynamic Unit Testing



- Execution-based unit testing is referred to as dynamic unit testing.
- In this testing, a program unit is actually executed in isolation, and the outcomes of program execution are observed.
- However, this execution differs from ordinary execution in the following way:
 - A unit under test (UUT) is taken out of its actual execution environment
 - The actual execution environment is emulated by writing more code so that the unit and the emulated environment can be compiled together
 - The above compiled aggregate is executed with selected inputs

Dynamic Unit Testing



- The environment of a unit is emulated and tested in isolation.
- The caller unit is known as **test driver**
 - A *test driver* is a program that invokes the unit under test (UUT)
 - It provides input data to UUT and report the test result
- The emulation of the units called by the UUT are called **stubs**
 - Stub is a “dummy sub-program” that replaces a unit that is called by the UUT
- The **test driver** and the **stubs** are together called **scaffolding**
- The low-level design document provides guidance for selection of input test data

Dynamic Unit Testing

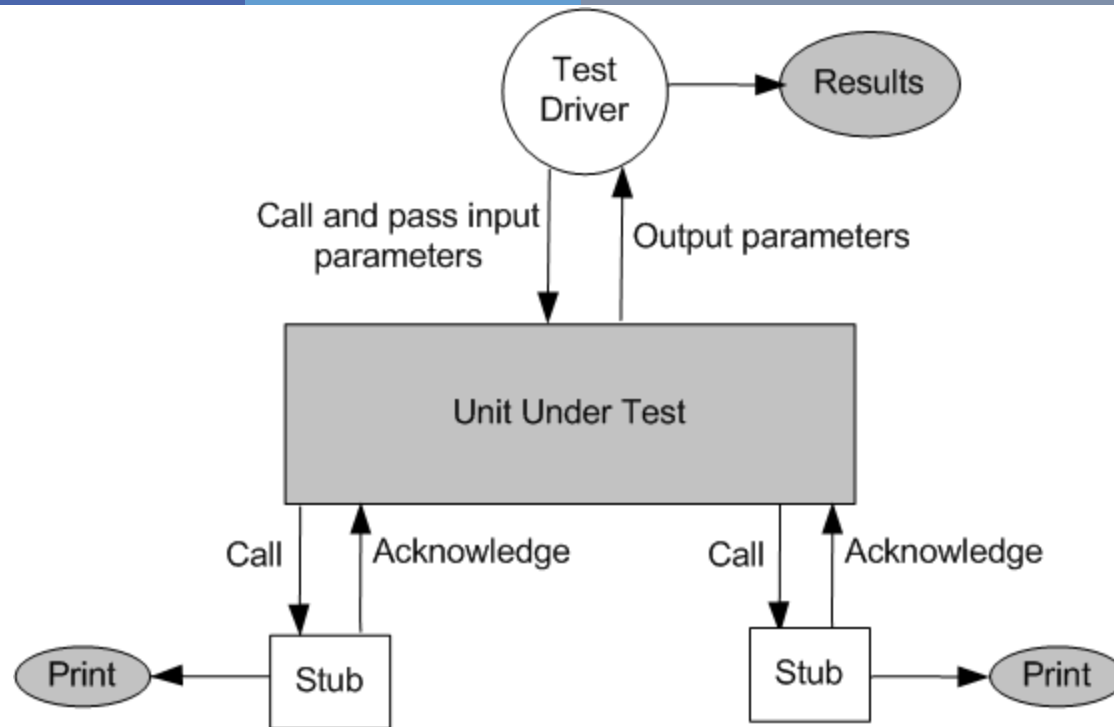


Figure : Dynamic unit test environment

Dynamic Unit Testing



- The low-level design document provides guidance for the selection of input test data that are likely to uncover faults. Selection of test data is broadly based on the following techniques:
 - **Control flow testing**
 - Draw a control flow graph (CFG) from a program unit
 - Select a few control flow testing criteria
 - Identify a path in the CFG to satisfy the selection criteria
 - Derive the path predicate expression from the selection paths
 - By solving the path predicate expression for a path, one can generate the data

Dynamic Unit Testing



- **Data flow testing**
 - Draw a data flow graph (DFG) from a program unit and then follow the procedure described in control flow testing
- **Domain testing**
 - *Domain errors* are defined and then test data are selected to catch those faults
- **Functional program testing**
 - Input/output domains are defined to compute the input values that will cause the unit to produce expected output values

Mutation Testing



- **Mutation testing** is a technique that focuses on measuring the adequacy of test data (or test cases).
- The original intention behind mutation testing was to expose and locate weaknesses in test cases.
 - Mutation testing is a way to measure the quality of test cases, and the actual testing of program is an added benefit
- Mutation testing is not a testing strategy like control flow or data flow testing. It should be used to supplement traditional unit testing techniques.

Mutation Testing



- A mutation of a program is a modification of the program created by introducing a single, small, legal syntactic change in the code.
- A modified program is called *mutant*.
- A mutant is said to be *killed* when the execution of test case causes it to fail. The mutant is considered to be *dead*.
- A mutant is an *equivalent* to the given program if it always produce the same output as the original program.
- A mutant is called *killable* or *stubborn*, if the existing set of test cases is insufficient to kill it.

Mutation Testing



- A *mutation score* for a set of test cases is the percentage of non-equivalent mutants *killed* by the test suite.

$$\text{Mutation Score} = (\# \text{ of Killed Mutants} / \text{Total number of Mutants}) * 100$$

- The *test suite* is said to be *mutation-adequate* if its mutation score is 100%
- Mutation testing may be used to judge the effectiveness of a test set. The test set should kill all the mutants

Mutation Testing



- Mutation testing is based upon seeding the implementation (original program) with a fault (*mutating* it), by applying a mutation operator, and determining whether testing identifies this fault.
- A mutated program is called a *mutant* and if a test case distinguishes between the mutant and the original program it is said to *kill* the mutant. Given a set of test cases, if no test case can distinguish between the mutant and the original program then the mutant is still *live*.
- Mutation testing is an effective approach for measuring the adequacy of the test cases. But, the main drawback is that the high cost of generating the mutants and executing each test case against that mutant program.

Mutation Testing



- Mutation testing makes two major **assumptions**:

1) Competent Programmer hypothesis

- Programmers are generally competent and they do not create “*random*” programs

2) Coupling effects

- Complex faults are coupled to simple faults in such a way that a test suite detecting simple faults in a program will detect most of the complex faults

Debugging



- The process of determining the cause of a defect and correcting it is known as *debugging*. *Debugging* occurs as a consequence of a test revealing a defect.
- After a program failure, the **programmer** identifies the corresponding fault and fixes it. Programmers can locate faults by debugging a unit.
- The objective of debugging is to precisely identify the cause of a failure. Given the symptoms of a problem, the purpose is to isolate & determine its specific cause.
- Debugging is a time consuming and error-prone process.
- Debugging involves a combination of systematic evaluation, intuition and a little bit of luck.

Debugging Techniques



- There are three approaches (techniques) to *debugging*
 - 1) **Brute force** : Most common but least efficient technique of debugging; “let the computer find the error” philosophy is used. Programmer manually searching through stack-traces, memory-dumps, log files, and so on, for traces of the error.
 - 2) **Cause elimination**: A listing of **causes** that may presumably have contributed to the error symptom is developed and tests are conducted to **eliminate** every.
 - 3) **Backtracking**: Programmer starts at a point in the code where a failure was observed and traces back the execution to the point where it occurred. Common debugging approach that can be used successfully in small programs.



Unit Testing in eXtreme Programming

1. Pick a requirement, i.e., a *story*
2. Write a test case that will verify a small part of the story and assign a fail verdict to it
3. Write the code that implement particular part of the story to pass the test
4. Execute all tests
5. Rework on the code, and test the code until all tests pass
6. Repeat step 2 to step 5 until the story is fully implemented

Unit Testing in eXtreme Programming

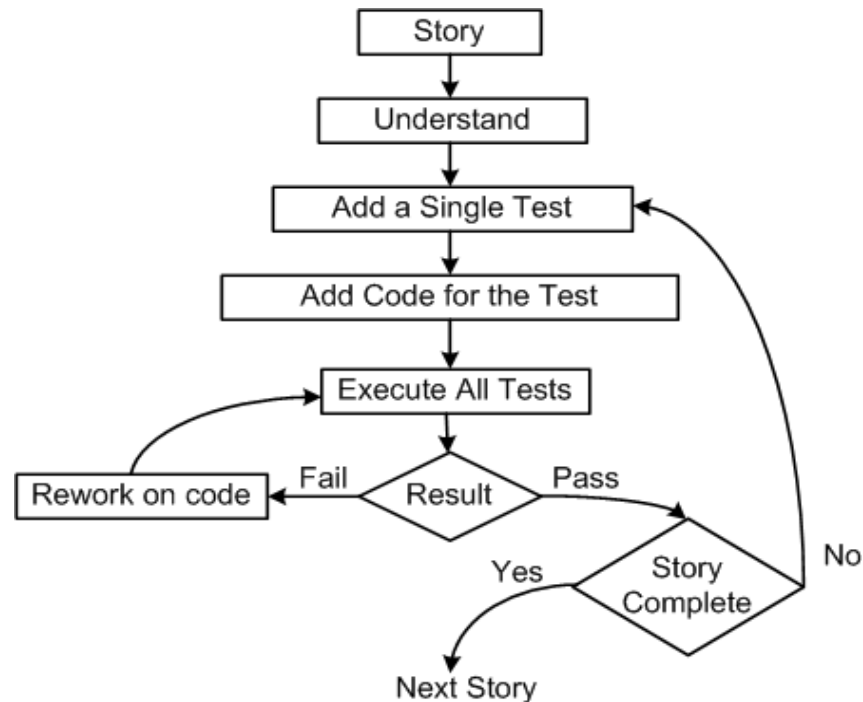


Figure : *Test-first* process in XP

Unit Testing in eXtreme Programming



- A *TDD (test driven development)* approach to code development is used in XP methodology. The key aspect of TDD approach is that a programmer writes low-level tests before writing production code. This is referred to as *test-first* in software development.
- In XP, a few unit tests are coded first, then a simple, partial system is implemented to pass the tests. Then, one more new unit test is created, and additional code is written to pass the new test, but not more, until a new unit test is created. The process is continued until nothing is left to test.
- In the XP process, unit tests are created prior to coding – this is known as *test-first*. The test-first approach sets up checks and balances to improve the chances of getting things right the first time.

Unit Testing in eXtreme Programming



■ Pair programming:

- In XP, the code is being developed by two programmers working together side by side.
- The two programmers sit side by side in front of the monitor. One person develops the code tactically and the other one inspects it methodically by keeping in mind the **story** they are implementing.



JUnit – A Framework for Unit Testing

- **JUnit:** It is a framework for performing unit testing of Java programs.
 - Other frameworks: NUnit (C#), CPPUNIT (C++), fUnit (Fortran)
- Intuitive steps to test a method in Java (Ex. Move() method of PlanetClass)
 - Create an object instance of PlanetClass. Call it Mars.
 - Select values of all input parameters of Move().
 - Compute the expected value to be returned by Move(). Let it be y.
 - Execute method Move() on Mars with the selected input values.
 - Let Move() return a value called z
 - Compare the **actual output (z)** returned by Move() with the **expected value (y)**.
 - If (z == y), Move() *passes* the test; otherwise it *fails*.
 - ← Report the result.

Tools For Unit Testing



- Programmers can benefit from using tools in unit testing by reducing testing time without sacrificing thoroughness.
- The well-known tools in everyday life are an editor, a compiler, an operating system, and a debugger.
- However, in some cases, the real execution environment of a unit may not be available to a programmer while the code is being developed. In such cases, an emulator of the environment is useful in testing and debugging the code.
- Other kinds of tools that facilitate effective unit testing are in the next slide...

Tools For Unit Testing



- **Code auditor** – used to check the quality of the software to ensure that it meets some minimum coding standard
- **Documenters** – read the source code and automatically generate descriptions and caller/callee tree diagram or data model from the source code
- **Interactive debuggers** – assist software developers in implementing different debugging techniques
- **Static code (path) analyzer** – identify paths to test based on the structure of code such as McCabe's cyclomatic complexity measure
- **Software inspection support** – can help schedule group inspection

Tools For Unit Testing



- **Test coverage analyzer** – measure internal test coverage, often expressed in terms of control structure of the test object, and report the coverage metric
- **Test data generator** – assist programmers in selecting test data that cause program to behave in a desired manner
- **Test harness** – support the execution of dynamic unit tests
- **Performance monitors** – The timing characteristics of the software components be monitored and evaluate by these tools
- **Network analyzers** – These tools have the ability to analyze the traffic and identify problem areas
- **Simulators and emulators** – used to replace the real software and hardware that are not currently available.
- **Version control** – A version control system provides functionalities to store a sequence of revisions of the software and associated information files under development

Books



- *Software Testing and Quality Assurance: Theory and Practice*, by Kshirasagar Naik, Priyadarshi Tripathy



References

1. *Software Quality Assurance: From Theory to Implementation*, by Daniel Galin
2. *Software Testing and Continuous Quality Improvement*, by William E. Lewis
3. *The Art of Software Testing*, by Glenford J. Myers, Corey Sandler and Tom Badgett
4. *Software Testing Fundamentals: Methods and Metrics* by Marnie L. Hutcheson